

Lección 23: APIs & Components Documentation

Documentación que Developers Realmente Usan

Agenda

- ¿Por qué documentar APIs y componentes?
 - OpenAPI/Swagger: Docs de API automáticas
 - Storybook: Librería viva de componentes
 - Documentación interactiva
 - Mejores prácticas
-

¿Por qué Documentar APIs? (1/2)

Sin documentación:

Developer: "¿Cómo creo un carrito?"

- Lee código fuente durante 30 minutos
- Prueba payloads aleatorios
- Obtiene errores 400
- Pregunta en Slack
- Finalmente funciona después de 2 horas

Tiempo perdido: 2 horas por developer

¿Por qué Documentar APIs? (2/2)

Con documentación:

Developer: Abre docs de la API
→ Ve ejemplo de request
→ Copia y adapta
→ Funciona en 5 minutos

Tiempo perdido: 5 minutos

ROI: La documentación ahorra 100x el tiempo que toma escribirla

Documentación de API: Qué Incluir

Elementos esenciales:

1. Endpoint: POST /api/cart
 2. Authentication: Bearer token required
 3. Request format: JSON body
 4. Request schema: CartItem[]
 5. Response format: JSON
 6. Response schema: Cart
 7. Error cases: 400, 401, 500
 8. Example request
 9. Example response
 10. Rate limits (if any)
-

Ejemplo de Documentación de API

POST /api/cart/items - Agregar item al carrito

Autenticación: Bearer token requerido

Request:

```
{  
  "productId": number,  
  "quantity": number // 1-99
```

```
}
```

Response 200 OK:

```
{
  "id": string,
  "items": CartItem[],
  "total": number // centavos
}
```

Errores:

- 400: Cantidad inválida
- 401: Sin autenticación
- 404: Producto no existe

OpenAPI / Swagger

¿Qué es?

Especificación estándar para describir REST APIs

- Escribes: openapi.yaml (estructura de tu API)
- Generas: Docs interactivas automáticas (Swagger UI)

Beneficios:

- ✓ Única fuente de verdad (spec = docs = validación)
- ✓ Docs nunca desactualizadas (generadas desde código)
- ✓ Interfaz "Try it out" (probar API en navegador)
- ✓ Genera SDKs de cliente automáticamente

Ejemplo de OpenAPI (Simplificado)

```
openapi: 3.0.0
info:
  title: Shopping Cart API

paths:
  /api/cart/items:
    post:
```

```
summary: Agregar item al carrito
security: [bearerAuth]
requestBody:
  schema:
    productId: integer
    quantity: integer (1-99)
responses:
  200: Item agregado (retorna Cart)
  400: Request inválido
  401: No autorizado
```

Swagger UI genera docs interactivas desde este YAML

Generando OpenAPI desde Código

Mejor práctica: Generar spec desde TypeScript

```
// Defines types (fuente de verdad)
type AddToCartRequest = {
  productId: number
  quantity: number
}

// Tool (tspec, tsoa, etc.) genera openapi.yaml automáticamente
```

Beneficios:

- ✓ Tipos TypeScript = fuente de verdad
- ✓ Docs NUNCA desincronizadas (auto-generadas)
- ✓ Validación en compile-time
- ✓ Autocompletado en IDE

Swagger UI: Docs Interactivos

Autogenerado desde OpenAPI:

Shopping Cart API

[Authorize 

POST /api/cart/items

Add item to cart

[Try it out] 🖱️ Click para probar

Request: { "productId": 42, "quantity": 2 }

[Execute] → Ejecuta request real

Response 200:

{ "id": "cart_abc123", "total": 241758 }

Developers prueban API directamente en navegador (sin Postman)

Documentación de Componentes: ¿Por Qué? (1/2)

Sin docs:

```
// El desarrollador ve el componente
<Button variant="???" size="???" onClick={???}>
  ???
</Button>
```

Preguntas:

- ¿Qué variantes existen?
- ¿Qué tamaños?
- ¿Qué props son requeridos?
- ¿Cómo se ve?
- ¿Hay ejemplos?

Lo que hace el developer:

- Lee código fuente
 - Prueba y error
 - Pregunta a compañero
-

Documentación de Componentes: ¿Por Qué? (2/2)

Con Storybook:

- Abre Storybook
- Ve todas las variantes
- Ve todos los props
- Ve ejemplos en vivo
- Copia código
- Funciona inmediatamente

Storybook: Librería de Componentes

¿Qué es?

- Documentación interactiva de componentes React
- Muestra todas las variantes de un componente
- Permite modificar props en vivo
- Aislado de la app (testing rápido)
- Guía de estilo viva

URL local: localhost:6006
Deploy: storybook.company.com

Beneficios:

- ✓ Diseñadores ven componentes reales (no mockups)
- ✓ Developers saben qué componentes existen (no duplicados)
- ✓ QA prueba componentes en aislamiento
- ✓ Documentación visual siempre actualizada

Ejemplo: Button Component

```
// Button.tsx
interface ButtonProps {
```

```
    variant: 'primary' | 'secondary' | 'danger'
    children: React.ReactNode
    onClick?: () => void
  }

export const Button = ({ variant, children, onClick }: ButtonProps) => {
  return (
    <button className={`btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  )
}
```

Ejemplo: Button Stories

```
// Button.stories.tsx
import { Button } from './Button'

export default {
  title: 'Components/Button',
  component: Button,
  tags: ['autodocs'], // Auto-genera página de docs
}

// Cada export = 1 variante visual
export const Primary = {
  args: { variant: 'primary', children: 'Add to Cart' },
}

export const Secondary = {
  args: { variant: 'secondary', children: 'Cancel' },
}

export const Danger = {
  args: { variant: 'danger', children: 'Delete' },
}
```

Storybook muestra las 3 variantes con controls interactivos

Storybook: Docs Autogenerados (1/2)

Desde JSDoc + TypeScript + Stories:

Button

Componente de botón principal para acciones importantes

Props

variant* 'primary' | 'secondary' | 'danger'
Variante del botón

size 'small' | 'medium' | 'large'
Tamaño del botón (default: 'medium')

children* ReactNode
Texto del botón

onClick () => void
Manejador de clic

disabled boolean
Estado deshabilitado

Stories

[Primary] [Secondary] [Danger] [Small] [Disabled]

Storybook: Docs Autogenerados (1/2)

Controls

variant: [primary ▼]
size: [medium ▼]
children: [Agregar al Carrito]
onClick: [action logged]
disabled: []

[Mostrar código]

```
<Button variant="primary" size="medium">  
  Agregar al Carrito  
</Button>
```


Desarrollo Dirigido por Documentación

Flujo tradicional:

1. Escribir componente
 2. Usar en app
 3. Encontrar bugs
 4. Documentar (tal vez)
- ✗ Documentación = afterthought

Flujo Documentation-Driven:

1. Escribir story (requirements)
 2. Implementar componente
 3. Probar en aislamiento
 4. Usar en app
- ✓ Documentación = guía de diseño

Beneficios:

- Requirements claros desde inicio
- Casos edge descubiertos temprano
- Docs nunca olvidadas
- Stories = tests visuales

Shopping Cart: CartItem Stories

Documentar todos los estados:

```
// Estados normales
export const Default = {
  args: { item: { name: 'Laptop', price: 999, quantity: 2 } },
}

// Estados edge case
export const SinStock = {
  args: { item: { ...Default.args.item, inStock: false } },
}
```

```
export const CantidadMaxima = {  
  args: { item: { ...Default.args.item, quantity: 99 } },  
}
```

Beneficio: Descubres edge cases ANTES de usuarios

Documentación Interactiva

Controles en vivo:

Storybook genera UI controls automáticamente:

```
quantity: [2] [▲] [▼] → Cambiar en vivo  
variant: [primary ▼] → Ver todas las opciones  
disabled: [ ] → Toggle true/false
```

- Componente se actualiza en tiempo real
- Designers pueden probar sin tocar código

Addon Actions: Loguea interacciones (clicks, cambios)

Addon a11y: Verifica accesibilidad automáticamente

- Contraste de colores
 - Labels ARIA
 - Navegación por teclado
-

API + Docs de Componentes Juntos

Sitio de documentación unificado:

```
docs.company.com  
├─ /api → OpenAPI / Swagger UI  
│  ├─ /cart  
│  ├─ /products  
│  └─ /auth  
├─ /components → Storybook  
│  ├─ /Button  
│  └─ /CartItem
```

```
|   └─ /Form
└─ /guides      → User guides
    └─ Getting started
    └─ Authentication
    └─ Testing
└─ /architecture → ADRs
    └─ 001-zustand
    └─ 002-vitest
```

¡Un solo lugar para toda la documentación!

Mejores Prácticas

1. Mostrar, no decir:

- ✗ Texto: "Button tiene 3 variantes: primary, secondary, danger"
- ✓ Ejemplos visuales: 3 stories mostrando cada variante

2. Documentar casos extremos:

```
export const EdgeCases = () => (  
  <>  
    <CartItem quantity={1} /> // Mínimo  
    <CartItem quantity={99} /> // Máximo  
    <CartItem inStock={false} /> // Sin stock  
  </>  
)
```

3. Ejemplos simples:

- ✗ args: mockCompleteCartWithDiscounts[0]
- ✓ args: { name: 'Laptop', price: 999, quantity: 1 }

4. Visual regression testing: Chromatic captura screenshots automáticamente

Herramientas

Documentación de API:

- **Swagger UI:** Docs interactivas OpenAPI
- **Redoc:** Docs OpenAPI elegantes
- **tspec/tsoa:** Genera OpenAPI desde TypeScript

Documentación de Componentes:

- **Storybook:** Librería de componentes (más popular)
- **Docz:** Alternativa basada en MDX

Visual Testing:

- **Chromatic:** Screenshot regression testing

Comandos:

```
pnpm storybook          # Iniciar Storybook (localhost:6006)
pnpm build-storybook    # Build docs estáticos
```

Ejercicio 1: Storybook Story para Component Library

Prompt:

Actúa como un design systems engineer creando component documentation con S

CONTEXT0: Storybook = living component library (documenta componentes con e interactivos). Stories = ejemplos de uso del componente con diferentes prop object: configura component title, args, argTypes (controls en UI). Story o ejemplo específico del component con props predefinidos. args property: val props del story. Interactive controls: Storybook genera UI para modificar a real-time. Documentation-driven development: escribir stories ANTES de impl (clarifica requirements). Component isolation: testear componentes sin app (faster feedback). Living documentation: docs se actualizan cuando componen (NO documentation drift). tags: ['autodocs'] genera página de docs automáti TypeScript types.

TAREA: Crea Storybook story para Button component con 3 variantes visuales.

INSTALACIÓN:

- Comando: `pnpm add -D @storybook/react @storybook/react-vite`
- Setup: `npx storybook init`
- Librería: `Component documentation tool`

IMPLEMENTACIÓN:

- Archivo: `src/shared/components/Button/Button.stories.tsx` (crear si no existe)
- Component: `Button` con props `variant`, `label`, `onClick`, `disabled`
- 3 Stories: `Primary` (`variant='primary'`), `Secondary` (`variant='secondary'`),

STORY STRUCTURE:

```
import type { Meta, StoryObj } from '@storybook/react'
import { Button } from './Button'

const meta: Meta<typeof Button> = {
  title: 'Components/Button',
  component: Button,
  tags: ['autodocs']
}

export default meta
type Story = StoryObj<typeof Button>

export const Primary: Story = {
  args: { label: 'Add to Cart', variant: 'primary' }
}
```

TESTING EN STORYBOOK:

1. Ejecutar: `pnpm storybook`
2. Abrir: `<http://localhost:6006>`
3. Navegar: `Components` → `Button` en sidebar
4. Verificar:
 - 3 stories visibles: `Primary`, `Secondary`, `Disabled`
 - Controls panel: cambiar props interactivamente
 - Docs tab: documentation auto-generated con types
5. Probar controls: cambiar label text → component actualiza en real-time

VALIDACIÓN: 3 stories deben renderizar correctamente con interactive controls

Aprende: Storybook crea living documentation con ejemplos

interactivos que nunca se desactualizan

Ejercicio 2: OpenAPI Endpoint Documentation

Prompt:

Actúa como un API documentation specialist creando OpenAPI 3.0 especificatio

CONTEXT0: OpenAPI (ex-Swagger) = standard specification para REST APIs (mac
YAML/JSON). Single [source](#) of truth: spec genera docs, client SDKs, mock ser
structure: info (metadata), paths (endpoints), components (reusable schemas
cada endpoint con method (get, post, put, delete). requestBody: schema del
POST/PUT. responses: schemas por status code (200, 400, 401, 500). schema:
format ([type](#), properties, required, example). Swagger UI: genera interactiv
OpenAPI (try it button). Benefits: docs never out of sync si generadas desde
contract: frontend + backend acuerdan spec antes de implementation (API-fir
Validation: tools validan requests/responses contra spec.

TAREA: Documenta endpoint POST /api/cart con OpenAPI 3.0 specification.

IMPLEMENTACIÓN:

- Archivo: docs/api/openapi.yaml (crear si no existe)
- Endpoint: POST /api/cart (add item to cart)
- Request: { productId: string, quantity: number }
- Responses: 200 (success), 400 (invalid input), 401 (unauthorized)

OPENAPI STRUCTURE:

```
openapi: 3.0.0
info:
  title: Shopping Cart API
  version: 1.0.0

paths:
  /api/cart:
    post:
      summary: Add item to cart
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [productId, quantity]
              properties:
                productId:
```

```
      type: string
      example: "prod-abc123"
    quantity:
      type: integer
      minimum: 1
      maximum: 99
      example: 2
  responses:
    '200':
      description: Item added successfully
    '400':
      description: Invalid input (quantity out of range)
    '401':
      description: Authentication required
```

VALIDACIÓN CON SWAGGER EDITOR:

1. Ir a: <<https://editor.swagger.io/>>
2. Pegar openapi.yaml content
3. Verificar:
 - No syntax errors (right panel muestra docs, NO errors)
 - Docs rendered correctly con description + example
 - Try it button available
4. Test: Click "Try it out" → ejecuta request mock

VALIDACIÓN: OpenAPI spec debe validar sin errores en Swagger Editor

Aprende: OpenAPI crea single source of truth para APIs,

generando docs interactivas automáticamente

Puntos Clave

1. **Docs de API:** Esenciales para productividad del developer
2. **OpenAPI:** Formato estándar, genera docs interactivos
3. **Generar desde Código:** Tipos como única fuente de verdad
4. **Swagger UI:** Interfaz interactiva “pruébalo”
5. **Docs de Componentes:** Storybook para librería de componentes viva
6. **Stories:** Ejemplos como documentación
7. **Dirigido por Documentación:** Escribir docs antes del código
8. **Interactivo:** Controles + actions para testing

9. **Accesibilidad:** Verificación a11y integrada

10. **Sitio Unificado:** API + Componentes + Guías juntos